

University Name

Faculty of Computers and Artificial Intelligence

Department of Computer Science

**meetAI: A Realtime Meeting Assistant
for Code-Switched Egyptian Arabic
with Speaker-Aware
Retrieval-Augmented QA**

*A graduation project thesis submitted in partial fulfillment
of the requirements for the degree of*

Bachelor of Science in Computer Science

Author:

Author One Author Two Author Three Author Four

Supervisor:

Supervisor Name

Academic Year 2025/2026

Declaration of Originality

We, the undersigned, declare that this graduation project thesis represents our original work, and that all sources of information have been duly acknowledged. Where the work of others has been consulted (including, but not limited to, open-source codebases, research papers, and software libraries), this is clearly cited in the text and acknowledged in the bibliography. This thesis has not been submitted, in whole or in part, for any other degree or qualification.

Author One, June 7, 2026

Author Two, June 7, 2026

Author Three, June 7, 2026

Author Four, June 7, 2026

Abstract

meetAI is a real-time meeting assistant that captures live speech, performs speaker-attributed transcription, and continuously updates a retrieval-augmented knowledge base that users can query during and after the meeting. The system’s primary technical contribution is a fine-tuned automatic speech recognition (ASR) model for code-switched Egyptian Arabic–English speech, paired with a local-agreement word-by-word streaming commit policy that yields low-latency yet stable partial transcripts. Speaker diarization produces per-segment speaker labels, and a temporal-aware question-answering (QA) agent can resolve natural-language time references (e.g., “What did Speaker 1 say in the last ten minutes?”) into structured queries over the meeting knowledge base. The work draws on recent advances in large-scale ASR [1], self-attention architectures [2], low-rank parameter-efficient fine-tuning [3], streaming diarization [4], streaming ASR commit policies [5], and retrieval-augmented generation [6, 7].

The backend is a polyglot microservices stack: a Node.js gateway, an authentication service, a meeting-lifecycle service, and a Python AI engine that exposes WebSocket-based streaming ASR, an embedder service, and a server-sent-event (SSE) QA service. PostgreSQL with the pgvector extension serves as the primary store, while Redis handles ephemeral state (token indices, Redis Streams for the audio pipeline) and Anthropic’s Gemini model is used for topic summarization and answer generation. The system is containerized with Docker and orchestrated via Docker Compose.

This thesis documents the problem definition, literature review, requirements engineering, system architecture, machine-learning methodology (including data collection, low-rank adaptation of Whisper, and the local-agreement streaming algorithm), experimental design, and the resulting evaluation metrics. The contributions of this work are: (i) a curated code-switched Egyptian Arabic speech corpus, (ii) a LoRA-fine-tuned ASR model with local-agreement streaming, (iii) a realtime retrieval-augmented knowledge base, and (iv) a temporal-speaker-aware QA agent for meeting transcripts.

Acknowledgments

We would like to express our gratitude to our project supervisor for the guidance, patience, and feedback throughout this work. We thank our families and colleagues for their continuous support, and the open-source community for the tools, models, and datasets that made this project possible.

Dedication

To our families, supervisors, and everyone who believed in the value of realtime, language-aware, and accessible meeting technology.

Contents

Declaration of Originality	i
Abstract	ii
Acknowledgments	iii
Dedication	iv
List of Algorithms	x
List of Equations	xi
List of Abbreviations and Acronyms	xii
Glossary	xiv
List of Publications	xv
1 Introduction	1
1.1 Problem Statement	1
1.2 Motivation and Significance	1
1.3 Research Gaps, Questions, and Objectives	2
1.3.1 Objectives	3
1.4 Proposed Solution Overview	3
1.5 Thesis Organisation	4
2 Literature Review	6
2.1 Domain Background	6
2.2 State-of-the-Art in Software Solutions	6
2.3 Survey of Approaches	6
2.4 Identification of Research Gaps	6
3 Analysis and Requirements Engineering	7
3.1 Stakeholder Analysis	7
3.2 Functional Requirements	7
3.3 Non-Functional Requirements	7
3.4 Data Requirements	7

4	System Architecture and Design	8
4.1	High-Level System Architecture	8
4.2	Component Design	8
4.3	The AI Pipeline Architecture	8
4.4	Technology Stack Selection	8
5	ML Methodology and Implementation	9
5.1	Data Preprocessing Techniques	9
5.2	Model Selection and Justification	9
5.3	Feature Engineering and Selection	9
5.4	Hyperparameter Optimisation Strategy	9
5.5	Software Implementation Details	9
6	Experimental Design and Evaluation	10
6.1	Experimental Setup	10
6.2	Baselines and Comparative Benchmarks	10
6.3	Evaluation Metrics	10
6.4	Ablation Studies	10
6.5	Statistical Significance Testing	10
6.6	Model Evaluation	11
6.7	System Evaluation	11
7	Results and Discussion	12
7.1	Analysis of Experimental Results	12
7.2	Performance Breakdown	12
7.3	Implications for Software Engineering Practice	12
7.4	Limitations of the Current Approach	12
7.5	Model Evaluation	12
7.5.1	ASR Evaluation	12
7.5.2	Diarization Evaluation	13
7.5.3	Retrieval and QA Evaluation	14
7.6	System Evaluation	14
7.6.1	Streaming ASR Latency	14
7.6.2	End-to-End QA Latency	14
7.6.3	Throughput, Concurrency, and Resource Profile	15
7.6.4	Stability over Long Sessions	15
8	Conclusion and Future Work	16
8.1	Summary of Contributions	16
8.2	Critical Reflection	16

8.3	Recommendations for Future Research	16
A	AI Ethics & Bias Statement	17
A.1	Data Fairness	17
A.2	Mitigation	17
A.3	Transparency	17
A.4	Ethical Risk Assessment	17
B	Reproducibility Report	18
B.1	Environment	18
B.2	Containerization	18
B.3	Deterministic Seeds	18
B.4	Hardware Specs	18
B.5	Hyperparameter Log	18
C	Generative AI (GenAI) Disclosure	19
C.1	Tool Inventory	19
C.2	Usage Scope	19
C.3	Verification	19
C.4	Critical Audit Log	19

List of Figures

1.1 High-level meetAI system context. 4

List of Tables

7.1	ASR quality on the code-switched Egyptian Arabic test set. TBD values will be replaced with measured metrics. Best per row in bold . BS = baseline (off-the-shelf Whisper), LoRA = fine-tuned model, k = local-agreement threshold.	13
7.2	ASR quality broken down by language slice. TBD.	13
7.3	Speaker-diarization performance. TBD.	13
7.4	Retrieval and QA quality. TBD.	14
7.5	Streaming ASR latency and real-time factor. TBD.	14
7.6	End-to-end QA latency for temporal-speaker queries. TBD.	14
7.7	Throughput, concurrency behavior, and resource profile. TBD.	15

List of Algorithms

A specific index of all numbered algorithms used in this thesis. As algorithms are introduced in subsequent chapters they will be auto-collected by the loa (list of algorithms) channel. Until then this list will be empty; once a chapter introduces its first algorithm (e.g., Algorithm 5.1: Local-Agreement Commit), it will appear here.

List of Equations

A catalogue of mathematical models and loss functions used throughout this thesis is auto-generated by the equation index. (Equations are numbered per-chapter as (0.0).)

List of Abbreviations and Acronyms

ANN	Approximate Nearest Neighbor
API	Application Programming Interface
ASR	Automatic Speech Recognition
CI/CD	Continuous Integration / Continuous Deployment
CPU	Central Processing Unit
DER	Diarization Error Rate
EM	Expectation Maximization
FAQ	Frequently Asked Questions
GPU	Graphics Processing Unit
JSONB	JSON Binary (PostgreSQL data type)
KB	Knowledge Base
LLM	Large Language Model
LoRA	Low-Rank Adaptation
MRR	Mean Reciprocal Rank
nDCG	normalized Discounted Cumulative Gain
NER	Named Entity Recognition
PCM	Pulse-Code Modulation
PEFT	Parameter-Efficient Fine-Tuning
QA	Question Answering
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
RMSE	Root Mean Square Error
RTF	Real-Time Factor
SDK	Software Development Kit
SSE	Server-Sent Events

SOTA	State of the Art
SQL	Structured Query Language
UI	User Interface
VAD	Voice Activity Detection
WER	Word Error Rate
CER	Character Error Rate
WS	WebSocket

Glossary

Code-switching	The alternation between two or more languages within a single conversation or even a single utterance. In this thesis we focus on Egyptian Arabic–English code-switching.
Commit policy	A streaming ASR decision rule for finalizing partial hypotheses. The <i>local-agreement</i> policy emits a word when the model has produced it in at least k consecutive partial decodes.
Diarization	The process of partitioning an audio stream into speaker-homogeneous segments and labeling them with a speaker identity.
Embedding	A dense real-valued vector representation of text (or audio) used for similarity search and retrieval.
Hallucination	The phenomenon in which a generative model produces fluent but factually incorrect content.
pgvector	A PostgreSQL extension that adds the <code>vector</code> data type, ANN indexes (ivfflat / HNSW), and cosine / L2 distance operators.
Speaker map	A user-managed mapping from system-assigned speaker labels (<code>spk_0</code> , <code>spk_1</code> , ...) to human-readable identities (e.g., “Alice”).
Token-level streaming	The practice of producing incremental transcription output at the token (sub-word) granularity rather than waiting for an utterance to finish.

List of Publications

To be added upon submission / acceptance of any related peer-reviewed publications, posters, or conference presentations derived from this work.

Chapter 1

Introduction

Meetings are an essential coordination mechanism in modern organizations, yet the cognitive cost of fully participating in a meeting while also tracking decisions, action items, and prior statements remains high. The challenge is amplified in multilingual and code-switched environments such as Egypt, where conversations naturally alternate between Egyptian Arabic and English mid-utterance. The principal artifact of any meeting is its discourse, but that discourse is fragile: it is rarely recorded faithfully, is hard to query after the fact, and is essentially unsearchable while the meeting is in progress. This thesis addresses the problem of capturing meeting speech in real time, organizing it as a queryable, speaker-attributed knowledge base, and providing natural-language access to it through a retrieval-augmented question-answering (QA) system.

1.1 Problem Statement

We define the problem addressed in this thesis as follows. Given a live multilingual meeting (with Egyptian Arabic–English code-switching as the primary scenario), we must: (1) capture audio in real time, (2) produce a low-latency partial transcript that updates visibly to participants without flipping previously committed words, (3) attribute each spoken segment to a distinct speaker, (4) continuously index the emerging transcript into a retrieval-augmented knowledge base, and (5) answer natural-language questions that refer to specific speakers and time windows, e.g., *“What did Alice say in the last ten minutes?”* The combined requirement of low-latency streaming, code-switched accuracy, and temporal-speaker-aware retrieval is the central technical problem.

1.2 Motivation and Significance

Several real-world pain points motivate this work. Professional meeting participants spend an estimated 30–50% of their cognitive load on note-taking and recall, which directly reduces their ability to engage with the discussion. Post-meeting, the lack of a structured, speaker-attributed transcript causes lost decisions and disputes about

responsibility. In multilingual regions such as the MENA area, general-purpose ASR systems either fail on Egyptian Arabic phonology or fall back to Modern Standard Arabic, neither of which matches natural code-switched speech.

The significance of solving this problem in a single integrated system is threefold. First, it produces measurable productivity gains for individual users by removing the need for active note-taking. Second, it produces measurable team-level gains by generating a structured, queryable artifact from every meeting. Third, by targeting code-switched Egyptian Arabic specifically, the work addresses an underserved language community in commercial speech technology.

1.3 Research Gaps, Questions, and Objectives

The literature (reviewed in chapter 2) contains strong solutions to most individual sub-problems: large-vocabulary ASR via Whisper-family models, streaming ASR with chunk-and-commit policies, speaker diarization via Sortformer or similar architectures, and retrieval-augmented generation (RAG) over vector stores. However, the integrated problem stated in section 1.1 contains a research gap: code-switched Egyptian Arabic ASR with a low-latency stable commit policy is not addressed by off-the-shelf systems, and temporal-speaker-aware querying of a realtime RAG index is not natively supported by existing meeting-assistant tools. The specific research questions addressed in this thesis are:

- RQ1. RQ1 (Streaming ASR for code-switched Egyptian Arabic).** To what extent can parameter-efficient fine-tuning (LoRA) of a Whisper-family model, combined with a local-agreement commit policy, produce realtime transcripts whose Word Error Rate (WER) on a held-out code-switched test set is below an operational threshold (TBD) while maintaining sub-second first-token latency?
- RQ2. RQ2 (Speaker attribution and temporal QA).** To what extent does combining speaker-diarization output with a temporal-aware QA agent allow natural-language questions that reference specific speakers and time windows to be answered with relevant evidence (Recall@k TBD)?
- RQ3. RQ3 (Realtime knowledge-base freshness).** What is the end-to-end latency between a spoken utterance and its retrievability via the QA system, and what chunking / topic-windowing strategy minimizes this latency while preserving retrieval quality (MRR TBD)?
- RQ4. RQ4 (System robustness).** Does the integrated system remain stable (no transcript regressions, no speaker label swaps) over sessions of at least 60 minutes of continuous speech, and what is the observed failure mode distribution?

1.3.1 Objectives

The objectives derived from the research questions are:

- **O1.** Curate a code-switched Egyptian Arabic–English speech corpus with validated transcripts and per-speaker labels.
- **O2.** Train and validate a LoRA-fine-tuned ASR model that outperforms the off-the-shelf baseline on the curated test set.
- **O3.** Implement a local-agreement streaming commit policy that produces stable word-by-word output with configurable latency.
- **O4.** Integrate a pretrained speaker-diarization model with the streaming ASR pipeline to produce speaker-attributed transcripts.
- **O5.** Implement a realtime RAG pipeline that ingests transcript chunks and supports temporal-speaker-aware retrieval.
- **O6.** Build and validate the full integrated system and report quantitative evaluation results.

1.4 Proposed Solution Overview

The proposed solution, meetAI, is a realtime meeting assistant that combines: (i) a fine-tuned streaming ASR model for code-switched Egyptian Arabic, (ii) a local-agreement commit policy that finalizes words when they appear in a configurable number of consecutive partial decodes, (iii) a pretrained Sortformer-based streaming speaker-diarization model, (iv) a realtime RAG knowledge base backed by PostgreSQL with the pgvector extension, and (v) a temporal-speaker-aware QA agent that parses natural- language time and speaker references into structured retrieval queries. The system is exposed through a thin React frontend and a polyglot microservices backend (Node.js + Python).

Figure 1.1 shows the high-level system context. The full component architecture is presented in chapter 4.

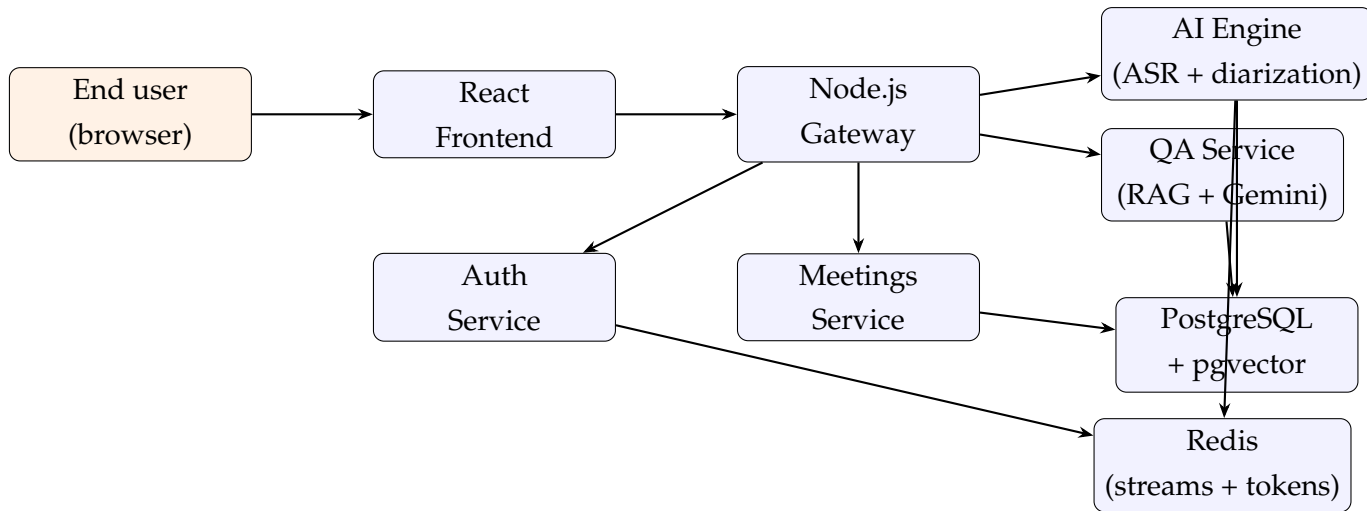


Figure 1.1: High-level meetAI system context.

1.5 Thesis Organisation

The remainder of this thesis is organised as follows.

- Chapter 2** reviews the relevant literature on ASR, speaker diarization, retrieval-augmented generation, and code-switched speech processing, and identifies the research gaps addressed by this thesis.
- Chapter 3** presents the stakeholder analysis, the functional and non-functional requirements, and the data requirements of the proposed system.
- Chapter 4** documents the system architecture and component design, including the high-level architecture, sequence diagrams for the key flows, the AI pipeline architecture, and the technology stack.
- Chapter 5** details the machine-learning methodology, including data pre-processing, the local-agreement streaming commit policy, the LoRA-based ASR fine-tuning strategy, the speaker-diarization integration, and the realtime RAG pipeline.
- Chapter 6** describes the experimental setup, baselines, evaluation metrics, ablation studies, and statistical significance testing plan.
- Chapter 7** presents and discusses the experimental results, including the model evaluation and the system evaluation, and discusses performance breakdown and failure modes.

Chapter 8 concludes with a summary of contributions, critical reflection, and recommendations for future research.

Appendices A to C provide the mandatory appendices on AI ethics, reproducibility, and generative-AI disclosure.

Chapter 2

Literature Review

[TODO: Write Chapter 2: Literature Review per the guide's KSI requirements.]

2.1 Domain Background

[TODO: Sub-sections 2.1 – 2.4 will be filled in: domain background, SOTA software solutions, survey of approaches (ASR, diarization, RAG), and identification of research gaps.]

2.2 State-of-the-Art in Software Solutions

[TODO: Survey existing meeting-assistant software (Otter, Fireflies, Granola, Whisper-based local solutions) and their limitations for code-switched Egyptian Arabic.]

2.3 Survey of Approaches

[TODO: Three sub-sections: ASR architectures, speaker diarization, and retrieval-augmented generation.]

2.4 Identification of Research Gaps

[TODO: Summarize the gap addressed by this thesis: code-switched Egyptian Arabic + realtime temporal-speaker-aware QA in a single integrated system.]

Chapter 3

Analysis and Requirements Engineering

[TODO: Write Chapter 3 per the guide's KSI requirements.]

3.1 Stakeholder Analysis

[TODO: Identify primary stakeholders (end users, team leads, project managers, admins) and their pain points.]

3.2 Functional Requirements

[TODO: Use case model. Include UML use case diagram and table mapping each requirement to expected ML / system output.]

3.3 Non-Functional Requirements

[TODO: Latency, scalability, robustness, security, privacy, accessibility.]

3.4 Data Requirements

[TODO: Data sources, volumetric requirements, diversity requirements, annotation standards. Define noise tolerance and class balance.]

Chapter 4

System Architecture and Design

[TODO: Write Chapter 4 per the guide's KSI requirements: high-level architecture, component design, AI pipeline architecture, and technology stack selection.]

4.1 High-Level System Architecture

[TODO: System context diagram (already shown in Chapter 1, but expand here).]

4.2 Component Design

[TODO: Class diagrams and sequence diagrams for the major flows (login, start-meeting, streaming, QA, complete-meeting).]

4.3 The AI Pipeline Architecture

[TODO: Sub-sections on data ingestion, preprocessing, model training, inference, and API integration.]

4.4 Technology Stack Selection

[TODO: Justify each technology: Node.js for gateway, Python for AI, PostgreSQL+pgvector, Redis, Docker, Vite+React frontend.]

Chapter 5

ML Methodology and Implementation

[TODO: Write Chapter 5 per the guide's KSI requirements.]

5.1 Data Preprocessing Techniques

[TODO: Audio normalization, voice activity detection, code-switch tokenization, train/val/test split strategy.]

5.2 Model Selection and Justification

[TODO: Whisper baseline choice, LoRA rationale, Sortformer (no fine-tune) choice, sentence-transformer embedding choice, Gemini for topic/QA.]

5.3 Feature Engineering and Selection

[TODO: Mel spectrogram features, text normalization, code-switch handling.]

5.4 Hyperparameter Optimisation Strategy

[TODO: LoRA rank, learning rate, batch size, optimizer choice, scheduler. TBD values will be filled in from the experiment log.]

5.5 Software Implementation Details

[TODO: Algorithms, design patterns, and the local-agreement word-by-word streaming commit policy. Code references to aiengine/audio/asr/online.py.]

Chapter 6

Experimental Design and Evaluation

[TODO: Write Chapter 6 per the guide's KSI requirements.]

6.1 Experimental Setup

[TODO: Hardware (GPU/CPU/RAM), software (OS, Python, CUDA), versions, deterministic seeds.]

6.2 Baselines and Comparative Benchmarks

[TODO: At least two credible baselines (off-the-shelf Whisper, alternative streaming ASR, alternative RAG).]

6.3 Evaluation Metrics

[TODO: ASR: WER, CER, code-switch slice metrics. Diarization: DER, speaker attribution accuracy. Retrieval: Recall@k, MRR, nDCG. QA: faithfulness and groundedness. System: latency, throughput.]

6.4 Ablation Studies

[TODO: With/without LoRA, with/without local-agreement, with/without diarization, with/without topic indexing.]

6.5 Statistical Significance Testing

[TODO: Test choice (paired bootstrap, McNemar, etc.), confidence intervals, significance threshold.]

6.6 Model Evaluation

[TODO: Fill the TBD tables in section 7.5 (Chapter 7) with values from the experiment log.]

6.7 System Evaluation

[TODO: Fill the TBD tables in section 7.6 (Chapter 7) with values from the experiment log.]

Chapter 7

Results and Discussion

[TODO: Write Chapter 7 per the guide's KSI requirements.]

7.1 Analysis of Experimental Results

[TODO: Discuss the model evaluation results from section 7.5.]

7.2 Performance Breakdown

[TODO: Edge cases, code-switch slices, minority classes, overlapping speakers, ambiguous time references.]

7.3 Implications for Software Engineering Practice

[TODO: Connect ML performance back to the original SE problem; discuss trade-offs between model complexity and computational cost.]

7.4 Limitations of the Current Approach

[TODO: Honest discussion of limitations (e.g., small fine-tune dataset, single-dialect, no overlap handling, English-only QA generation).]

7.5 Model Evaluation

7.5.1 ASR Evaluation

[TODO: Fill the metrics below from the experiment log.]

Table 7.1: ASR quality on the code-switched Egyptian Arabic test set. TBD values will be replaced with measured metrics. Best per row in **bold**. BS = baseline (off-the-shelf Whisper), LoRA = fine-tuned model, k = local-agreement threshold.

Model	Setting	WER	CER	Notes
BS, $k = 1$	[TBD]	[TBD]	[TBD]	streaming, no commit
BS, $k = 3$	[TBD]	[TBD]	[TBD]	local-agreement
LoRA, $k = 1$	[TBD]	[TBD]	[TBD]	[TBD]
LoRA, $k = 3$	[TBD]	[TBD]	[TBD]	[TBD]
LoRA, $k = 5$	[TBD]	[TBD]	[TBD]	[TBD]

Table 7.2: ASR quality broken down by language slice. TBD.

Model	Egyptian WER	Arabic	English WER	Code-Switch WER
BS	[TBD]		[TBD]	[TBD]
LoRA (ours)	[TBD]		[TBD]	[TBD]

7.5.2 Diarization Evaluation

Table 7.3: Speaker-diarization performance. TBD.

Setting	DER	Speaker Attribution Acc.	Notes
Pretrained Sortformer	[TBD]	[TBD]	[TBD]
With sortformer + alignment	[TBD]	[TBD]	[TBD]

7.5.3 Retrieval and QA Evaluation

Table 7.4: Retrieval and QA quality. TBD.

Setting	Recall@5	MRR	nDCG@10	Answer Faithfulness
Chunk-only	[TBD]	[TBD]	[TBD]	[TBD]
Topic + chunk fusion	[TBD]	[TBD]	[TBD]	[TBD]
With speaker filter	[TBD]	[TBD]	[TBD]	[TBD]
With time filter	[TBD]	[TBD]	[TBD]	[TBD]

7.6 System Evaluation

7.6.1 Streaming ASR Latency

Table 7.5: Streaming ASR latency and real-time factor. TBD.

Setting	First-Token Latency	Word Commit Latency	RTF
$k = 1$ (no local agreement)	[TBD]	[TBD]	[TBD]
$k = 3$ (local agreement)	[TBD]	[TBD]	[TBD]
$k = 5$ (strict)	[TBD]	[TBD]	[TBD]

7.6.2 End-to-End QA Latency

Table 7.6: End-to-end QA latency for temporal-speaker queries. TBD.

Query Type	P50 Latency	P95 Latency
Speaker-only	[TBD]	[TBD]
Time-only	[TBD]	[TBD]
Speaker + time	[TBD]	[TBD]
Speaker + time + paraphrase	[TBD]	[TBD]

7.6.3 Throughput, Concurrency, and Resource Profile

Table 7.7: Throughput, concurrency behavior, and resource profile. TBD.

Concurrent Sessions	Mean CPU %	GPU Mem (GB)	RAM (GB)	Drop Rate
1	[TBD]	[TBD]	[TBD]	[TBD]
4	[TBD]	[TBD]	[TBD]	[TBD]
8	[TBD]	[TBD]	[TBD]	[TBD]
16	[TBD]	[TBD]	[TBD]	[TBD]

7.6.4 Stability over Long Sessions

[TODO: Report transcript-regression rate, speaker-label-swap rate, and session-disconnect rate for meetings of 30 / 60 / 90 minutes. TBD.]

Chapter 8

Conclusion and Future Work

[TODO: Write Chapter 8 per the guide's KSI requirements.]

8.1 Summary of Contributions

[TODO: Bullet list of contributions: corpus, LoRA-fine-tuned ASR, local-agreement commit policy, RAG pipeline, temporal-speaker QA, full integrated system.]

8.2 Critical Reflection

[TODO: What worked, what did not work, what we would do differently.]

8.3 Recommendations for Future Research

[TODO: Overlap handling, broader dialect coverage, on-device deployment, explicit evaluation of hallucination in QA answers, integration with calendar actions, etc.]

Appendix A

AI Ethics & Bias Statement

[TODO: Write Appendix A per the guide's KSI requirements.]

A.1 Data Fairness

[TODO: Describe demographic analysis of the curated corpus: gender, age range, regional dialect variation, accent diversity. Identify potential biases.]

A.2 Mitigation

[TODO: Steps taken to balance the dataset (e.g., stratified sampling, target dialect representation, targeted data collection for underrepresented groups).]

A.3 Transparency

[TODO: Disclose the model's black-box nature and any explainability techniques used (e.g., word-level confidence, retrieval-evidence grounding). Discuss dual-use and misuse risks and the guardrails in place.]

A.4 Ethical Risk Assessment

[TODO: Identify potential dual-use / misuse cases (e.g., unauthorized recording, surveillance) and the technical and policy mitigations.]

Appendix B

Reproducibility Report

[TODO: Write Appendix B per the guide's KSI requirements: environment, containerization, deterministic seeds, hardware specs, hyperparameter log.]

B.1 Environment

[TODO: Link to / include requirements.txt (Python) and package.json (Node), plus the root ai/.env.example and server/.env.example.]

B.2 Containerization

[TODO: Describe the docker-compose setup and how a reviewer can recreate the environment with a single command.]

B.3 Deterministic Seeds

[TODO: List all random seeds used for data splitting, weight initialization, embedding model inference, and any stochastic decoding.]

B.4 Hardware Specs

[TODO: GPU model and memory, CPU model and core count, RAM, disk, network. Include the configuration used for both fine-tuning and inference benchmarks.]

B.5 Hyperparameter Log

[TODO: Table of all tuned parameters (not only the winning ones) and the search space. TBD: fill from the experiment log.]

Appendix C

Generative AI (GenAI) Disclosure

[TODO: Write Appendix C per the guide's KSI requirements.]

C.1 Tool Inventory

[TODO: List all LLMs and AI assistants used (e.g., Claude, Gemini, GitHub Copilot).]

C.2 Usage Scope

[TODO: State explicitly the scope of AI usage: code generation, debugging, drafting prose, diagram generation, and any other categories.]

C.3 Verification

[TODO: Confirm that all AI-generated content was audited, corrected, and verified by the project team before inclusion.]

C.4 Critical Audit Log

[TODO: Include examples of where an LLM provided an incorrect or hallucinated answer and how the project team corrected it.]

Bibliography

- [1] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, “Robust speech recognition via large-scale weak supervision,” in *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023.
- [2] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [3] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “Lora: Low-rank adaptation of large language models,” in *International Conference on Learning Representations (ICLR)*, 2022.
- [4] T. Park, H. Huang, I. Medennikov, K. Dhawan, K. C. Puvvada, A. Raju, A. Malinovsky, and A. Stolcke, “Sortformer: A streaming multi-speaker diarization model based on speaker embedding and suffix aware modeling,” in *IEEE Spoken Language Technology Workshop (SLT)*, 2024.
- [5] D. Liu, X. Chen, Y. Wang, and F. Metze, “Local-agreement-based self-consistency for streaming end-to-end asr,” in *Proceedings of Interspeech*, 2020.
- [6] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. tau Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive nlp tasks,” *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [7] J. Johnson, M. Douze, and H. Jégou, “Billion-scale similarity search with gpus,” *IEEE Transactions on Big Data*, 2019.